Stack Tecnológico

Frontend (El Garaje y la Interfaz de Usuario)

Framework: React (inicializado con Vite para mayor velocidad de compilación).

Arquitectura: PWA (Progressive Web Application). Esto permite desarrollar en web pero ofrecer al usuario la opción de instalar el sistema en su dispositivo (iOS/Android) con una experiencia nativa, sin pasar por las tiendas de apps en esta fase MVP.

Estilos y UI: TailwindCSS para un diseño rápido y responsivo.

Animaciones (Crítico): Framer Motion. Será la librería encargada de la fluidez a 60 FPS al hacer swipe para abrir la Caja de Importación y revelar los vehículos.

Gestión de Estado: Zustand o Redux Toolkit para mantener sincronizado el conteo de créditos y el inventario en tiempo real sin recargar la página.

Backend (El Motor del Juego)

Lenguaje y Framework: Python con FastAPI. La sintaxis de Python es ideal para programar de forma limpia el algoritmo estadístico (RNG) del Drop Rate. Además, FastAPI genera automáticamente la documentación de los endpoints (Swagger), lo que acelera las pruebas.

Base de Datos Relacional: MySQL (utilizando el motor SQLite). Garantiza la ejecución de bloqueos pesimistas (FOR UPDATE) requeridos para las transacciones ACID de los intercambios y el control de la economía.

ORM: SQLAlchemy (Python) para mapear las tablas y ejecutar consultas seguras previniendo inyecciones SQL.

Estructura del Código (Monorepo)

/racing-collection-mvp

```
├── /client               # Frontend: PWA en React
│   ├── /public           # Iconos de la PWA, manifest.json
│   ├── /src
│   │   ├── /assets       # Imágenes estáticas (Ej: siluetas de autos, logos)
│   │   ├── /components   # UI reutilizable (VehicleCard.jsx, CrateAnimation.jsx)
│   │   ├── /pages        # Vistas (Garage.jsx, Unboxing.jsx, Market.jsx, Sponsors.jsx)
│   │   ├── /store        # Lógica de estado global (Zustand/Redux)
```

```
│   │   └── /services        # Funciones fetch/axios que consumen la API
│   ├── package.json
│   └── vite.config.js
│
├── /server                  # Backend: API REST en Python (FastAPI)
│   ├── /app
│   │   ├── /api             # Rutas (endpoints) organizadas por módulo
│   │   │   ├── unboxing.py
│   │   │   ├── economy.py   # Lógica de los trailers y créditos
│   │   │   └── trades.py
│   │   ├── /core
│   │   │   ├── database.py  # Conexión a MySQL
│   │   │   └── rng_engine.py # Motor matemático de probabilidades
│   │   ├── /models          # Modelos SQLAlchemy (Tablas)
│   │   └── /schemas         # Modelos Pydantic (Validación de datos de entrada/salida)
│   ├── requirements.txt     # Dependencias de Python
│   └── main.py              # Archivo de arranque del servidor
│
├── /database                # Modelado y Scripts
│   ├── schema.sql           # DDL de creación de tablas
│   └── seed_vehicles.sql    # DML para insertar el catálogo inicial de vehículos
│
└── README.md                # Documentación del proyecto y comandos de inicio
```

Integración del Flujo de Trabajo

Para trabajar de manera eficiente en AntiGravity IDE:

1. Terminales Paralelas: Abre dos pestañas de terminal en tu IDE. Una apuntará a /client (ejecutando npm run dev) y la otra a /server (ejecutando uvicorn main:app --reload).

2. Desarrollo Desacoplado: Si estás programando el algoritmo matemático de apertura de sobres en /server/app/core/rng_engine.py, puedes probar la respuesta del servidor en el navegador (Swagger en localhost:8000/docs) antes de tocar el código de React.

3. Gestión de Assets: Las imágenes de los vehículos (incluyendo modelos específicos como los clásicos de los 90s) se guardarán inicialmente en /client/src/assets, pero a medida que el catálogo crezca, la estructura está lista para migrar esas URLs a un servicio en la nube y solo guardar las rutas en la base de datos MySQL